7. toArray() Methods — Quick Notes

toArray()

Purpose

Convert: Collection → Array

Two Versions

Method	Return Type
`toArray()`	`Object[]`
`toArray(T arr[])`	Typed Array

Version 1 — Object Array

Object arr[] = list.toArray();

Example:

List<Integer> list = new ArrayList<>();

list.add(10);
list.add(15);

Object arr[] = list.toArray();
Output: 10 15

Problem with Version 1

Need typecasting.

Example: Integer x = (Integer) arr[0];

Because: toArray()

Returns: Object[]

Version 2 — Typed Array

Integer arr[] = list.toArray(new Integer[0]);

Returns: Integer[]

No typecasting needed.

Example

```
List<Integer> list = new ArrayList<>();

list.add(10);
list.add(15);
list.add(20);

Integer arr[] = list.toArray(new Integer[0]);

for(Integer x : arr)
  System.out.println(x);
```

Output:

```
10
15
20
```

Why `new Integer[0]`?

Purpose:

- Tell datatype

new Integer[0]

Means: Create Integer array type

Size `0` is okay.

`toArray()` creates proper-sized array internally.

Existing Array Case

```
Integer arr[] = new Integer[5];

list.toArray(arr);
```

If array has enough space:

- Same array reused

Result: [10, 15, 20, null, null]

Important Point

If passed array:

- has enough space → reused
- doesn't have enough space → new array created

Works for Most Collections

Because method exists in:

Collection Interface

Works for:

- ArrayList
- LinkedList
- HashSet
- TreeSet
- ArrayDeque
- PriorityQueue

Order Depends on Collection

Collection	Order
ArrayList	Insertion order
TreeSet	Sorted order
HashSet	No fixed order
PriorityQueue	Heap order

Important Point

`toArray()` creates:

Separate Array

After conversion:

- Array changes NOT reflected in collection
- Collection changes NOT reflected in array

Difference from `Arrays.asList()`

toArray()	Arrays.asList()
Collection → Array	Array → List
Separate memory	Wrapper over same array
Independent	Shared changes